Bug Report

SmartInventoryApplication

Code Review — April 2026

CRITICAL	HIGH	MEDIUM	LOW
3	5	6	2

Executive Summary

This report documents 16 bugs discovered through static code analysis of the SmartInventoryApplication Spring Boot codebase. The application manages warehouse inventory with JWT-based authentication, H3 geospatial indexing, and event-driven low-stock alerting.

Three Critical and four High severity issues require immediate attention before any production deployment. The most serious findings are an open privilege escalation vulnerability on the registration endpoint, a role name inconsistency that may lock out admin users, and insufficient JWT key length enforcement.

Issue Summary

ID	Severity	Title	Component
BUG-001	CRITICAL	Open Registration Endpoint — Any User Can Self-Assign ADMIN Role	*AuthController / Security*
BUG-002	CRITICAL	Role Name Mismatch — ADMIN Users Are Permanently Locked Out	*SecurityConfig / UserService / Role enum*
BUG-003	CRITICAL	JWT Secret Read as UTF-8 Bytes — HS256 Key Too Short	*JwtUtil*
BUG-004	HIGH	No Global Exception Handler — Internal Errors Leak Stack Traces	*All Controllers*
BUG-005	HIGH	No Input Validation on Any DTO — Null and Invalid Data Accepted	*All DTOs / Controllers*
BUG-006	HIGH	Circular Dependency Between JwtFilter and UserService via @Autowired Setter	*JwtFilter / UserService / SecurityConfig*
BUG-007	HIGH	LowStockEvent Published Inside @Transactional Method — Alert May Save Before Stock Commits	*StockService / LowStockEventListener*
BUG-008	HIGH	H3Helper Uses a Static Initializer — Startup Crash If H3Core Fails, No Null Safety	*H3Helper / Warehouse entity*
BUG-009	MEDIUM	No Pagination on List Endpoints — Memory and Performance Risk	*All Controllers / Services*
BUG-010	MEDIUM	Password Returned in Register Response via User Entity	*AuthController*
BUG-011	MEDIUM	AnalyticsService.aggregateByH3() Loads Entire Stock Table — O(n) Memory	*AnalyticsService*
BUG-012	MEDIUM	spring.jpa.hibernate.ddl-auto=update in Production Configuration	*Configuration*
BUG-013	MEDIUM	LowStockEvent Fired on Exact Threshold — Duplicate Alerts on Repeated Updates	*StockService / LowStockEventListener*

BUG-014	**MEDIUM**	AlertController Missing From Security Rules — Unauthenticated Access Possible	*SecurityConfig / AlertController*
BUG-015	**LOW**	Warehouse Entity Has Two Duplicate @Table and @Entity Import Blocks	*Warehouse entity*
BUG-016	**LOW**	JwtUtil Uses Deprecated JJWT API (parserBuilder deprecated in newer versions)	*JwtUtil*

Detailed Bug Reports

BUG-001 CRITICAL

Open Registration Endpoint — Any User Can Self-Assign ADMIN Role

Component	AuthController / Security
File(s)	*AuthController.java*

Summary

The /api/auth/register endpoint accepts a 'role' query parameter with no authentication, no authorization, and no validation against the Role enum. Any anonymous caller can POST /api/auth/register?role=Admin and immediately obtain an admin account.

Steps to Reproduce

1. Send: POST /api/auth/register?role=Admin
2. Body: {"username":"hacker","password":"1234"}
3. Receive 200 OK with userId and role=Admin
4. Log in and gain full admin access to all endpoints

Root Cause

The "role" parameter uses @RequestParam(defaultValue = "Staff") with no @PreAuthorize or whitelist check. The Role enum (ADMIN, STAFF, VIEWER) is never consulted.

Relevant Code

```
public ResponseEntity<?> register(@RequestBody AuthRequest req,
    @RequestParam(defaultValue = "Staff") String role) {
    User user = userService.registerUser(req.getUsername(), req.getPassword(), role);
    ...
```

Recommended Fix

Remove the role parameter from the public endpoint. Either hardcode role="STAFF" for self-registration, or create a separate @PreAuthorize("hasRole('ADMIN')") endpoint for admin-assigned roles.

Impact

Full privilege escalation. Anyone on the internet can become admin.

BUG-002 CRITICAL

Role Name Mismatch — ADMIN Users Are Permanently Locked Out

Component	SecurityConfig / UserService / Role enum
File(s)	*SecurityConfig.java, UserService.java, Role.java*

Summary

Spring Security stores roles with the "ROLE_" prefix. UserService calls passwordEncoder.encode and saves role as-is (e.g. "Admin"). SecurityConfig uses hasRole("Admin") which resolves to ROLE_Admin, but the Role enum declares ADMIN (uppercase). The JwtUtil encodes the raw authority string from auth.getAuthorities() which is already "ROLE_Admin". The mismatch means hasRole() checks never match correctly.

Steps to Reproduce

1. Register: POST /api/auth/register (role="Admin")
2. Login and receive JWT
3. Call GET /api/warehouses — expect 200, receive 403
4. The authority stored is "ROLE_Admin" but Spring checks for "ROLE_Admin" — appears to match, but Role enum ADMIN vs string "Admin" causes inconsistency across the codebase

Root Cause

Three separate representations of role exist: the Role enum uses ADMIN, the register endpoint default is "Staff" (mixed case), SecurityConfig uses hasRole("Admin"). None are in sync. UserService.loadUserByUsername() concatenates "ROLE_" + user.getRole() so the actual authority depends entirely on whatever string was persisted.

Relevant Code

```
// Role enum
enum Role { ADMIN, STAFF, VIEWER }

// SecurityConfig
.hasRole("Admin")            // expects ROLE_Admin

// UserService.registerUser
.role(role)                  // stores raw param string (e.g. "Staff")

// UserService.loadUserByUsername
new SimpleGrantedAuthority("ROLE_" + user.getRole())
```

Recommended Fix

Standardize on the Role enum. Validate the incoming role parameter against Role.valueOf(role.toUpperCase()). Store role as enum name (uppercase). Change SecurityConfig to hasRole("ADMIN"), hasRole("STAFF"), hasRole("VIEWER").

Impact

Admins may be locked out of admin-only routes. Authorization is unpredictable based on what casing was used at registration time.

BUG-003 CRITICAL

JWT Secret Read as UTF-8 Bytes — HS256 Key Too Short

Component	JwtUtil
File(s)	*JwtUtil.java, application.properties*

Summary

Keys.hmacShaKeyFor(secret.getBytes()) requires the byte array to be at least 256 bits (32 bytes) for HS256. If the ${JWT_SECRET} environment variable is shorter than 32 characters, JJWT throws a WeakKeyException at startup. There is no validation, no minimum-length enforcement, and no error message to the developer. The application can also start successfully in some environments but silently accept weaker keys.

Steps to Reproduce

1. Set JWT_SECRET=short in environment
2. Start the application
3. Application throws WeakKeyException and fails to start — or if using an older JJWT version, starts with an insecure key

Root Cause

No minimum length validation on the JWT secret. Raw UTF-8 bytes are used directly, meaning non-ASCII characters inflate apparent length while byte length may still be insufficient.

Relevant Code

```
private Key getKey() {
    return Keys.hmacShaKeyFor(secret.getBytes()); // no length check
}
```

Recommended Fix

Add a @PostConstruct validation: assert secret.getBytes(StandardCharsets.UTF_8).length >= 32. Use StandardCharsets.UTF_8 explicitly. Consider using Keys.secretKeyFor(SignatureAlgorithm.HS256) for generated secrets.

Impact

Application startup failure in production if secret is short. Possible use of cryptographically weak JWT signing key.

BUG-004 HIGH

No Global Exception Handler — Internal Errors Leak Stack Traces

Component	All Controllers
File(s)	*ProductController.java, StockController.java, WarehouseController.java, AlertController.java*

Summary

All controllers call service methods that throw RuntimeException with messages like "Product not found: 42". Spring Boot default error handling will return a 500 Internal Server Error with the full exception message and potentially a stack trace (in non-production profiles). There is no @ControllerAdvice / @RestControllerAdvice, no custom error responses, and no HTTP 404 for missing resources.

Steps to Reproduce

1. Call GET /api/products/9999 (nonexistent product)
2. Receive HTTP 500 with body containing: 'Product not found: 9999'

3. In dev/default profile, full stack trace is exposed in response body

Root Cause

No @RestControllerAdvice exception handler. Service methods throw RuntimeException which maps to 500. There are no custom exception classes (e.g. ResourceNotFoundException) to map to 404.

Relevant Code

```
// ProductService.java
public Product getById(Long id) {
    return productRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Product not found: " + id));
    // Should be: throw new ResourceNotFoundException(...) -> HTTP 404
}
```

Recommended Fix

Create a ResourceNotFoundException extending RuntimeException. Add @RestControllerAdvice with @ExceptionHandler(ResourceNotFoundException.class) returning ResponseEntity with HTTP 404 and a structured error body.

Impact

Information disclosure (stack traces in production). All not-found scenarios return 500 instead of 404, breaking REST semantics and client error handling.

BUG-005 HIGH

No Input Validation on Any DTO — Null and Invalid Data Accepted

Component	All DTOs / Controllers
File(s)	*ProductRequest.java, StockUpdateRequest.java, BarcodeRequest.java, WarehouseRequest.java, AuthRequest.java*

Summary

No Jakarta Bean Validation annotations (@NotNull, @NotBlank, @Min, @Max, etc.) exist on any DTO, and no @Valid annotation is used in any controller method. This allows null fields, empty strings, negative prices, zero restock thresholds, and null product/warehouse IDs to reach the database layer.

Steps to Reproduce

1. POST /api/products with body: {"name": null, "price": -99.99, "restockThreshold": -5}
2. Product is created with null name and negative price
3. POST /api/stock/update with body: {"productId": null} → NullPointerException in service

Root Cause

DTOs are plain @Data Lombok classes with no validation. Controllers use @RequestBody without @Valid. Spring Boot's validation starter is presumably available but unused.

Relevant Code

```
// ProductRequest.java — no validation
@Data
public class ProductRequest {
    private String name;           // should be @NotBlank
    private Double price;          // should be @Positive
    private Integer restockThreshold; // should be @Min(0)
}

// Controller — missing @Valid
public ResponseEntity<Product> create(@RequestBody ProductRequest req)
```

Recommended Fix

Add @NotBlank, @NotNull, @Positive, @Min(0) etc. to all DTO fields. Add @Valid to all @RequestBody parameters. Add @MethodArgumentNotValidException handler in @RestControllerAdvice.

Impact

Data integrity violations, potential NullPointerExceptions surfaced as 500 errors, corrupt database records.

BUG-006 HIGH

Circular Dependency Between JwtFilter and UserService via @Autowired Setter

Component	JwtFilter / UserService / SecurityConfig
File(s)	*JwtFilter.java*

Summary

JwtFilter depends on UserService (which itself depends on PasswordEncoder from AppConfig). SecurityConfig depends on JwtFilter. Spring Boot can experience circular dependency or initialization ordering issues. The workaround using @Autowired setter injection (instead of constructor injection) with a comment "не final" (Russian: "not final") indicates this was a known problem patched with a fragile workaround rather than a proper fix.

Steps to Reproduce

1. Review the non-final userDetailsService field in JwtFilter
2. The setter injection exists solely to break a circular dependency cycle
3. If Spring Boot context initialization order changes, the filter may run with a null userDetailsService, causing NullPointerException on any authenticated request

Root Cause

SecurityConfig → JwtFilter → UserService → PasswordEncoder creates a dependency cycle. The @Lazy or setter pattern is a workaround, not a solution. The proper fix is architectural.

Relevant Code

```
// JwtFilter.java
private UserDetailsService userDetailsService; // <-- не final (comment in Russian)

public JwtFilter(JwtUtil jwtUtil) {
```

```
    this.jwtUtil = jwtUtil; // userDetailsService NOT injected here
}

@Autowired  // fragile setter injection to avoid circular dep
public void setUserDetailsService(UserDetailsService uds) { ... }
```

Recommended Fix

Extract a JwtTokenValidator component that only depends on JwtUtil. Have JwtFilter use ApplicationContext.getBean() lazily, or restructure to use @Lazy on the UserService dependency in JwtFilter.

Impact

Potential NullPointerException in JwtFilter if bean initialization order changes. Technical debt and fragile application context.

BUG-007 HIGH

LowStockEvent Published Inside @Transactional Method — Alert May Save Before Stock Commits

Component	StockService / LowStockEventListener
File(s)	*StockService.java, LowStockEventListener.java*

Summary

StockService.updateStock() is annotated @Transactional and publishes a LowStockEvent via applicationEventPublisher before the transaction commits. LowStockEventListener.handleLowStockEvent() is a synchronous @EventListener that saves a LowStockAlert to the database immediately. If the outer transaction rolls back after the event fires (e.g. due to a constraint violation in auditLogRepository.save()), the alert will have been persisted without a corresponding stock change.

Steps to Reproduce

1. Trigger a stock update that causes quantity <= restockThreshold
2. Event fires synchronously inside the transaction
3. LowStockAlert is saved to DB
4. Outer transaction rolls back due to a constraint violation
5. Alert exists in DB but no stock change occurred — orphaned alert

Root Cause

Spring's default @EventListener fires synchronously within the current transaction. @TransactionalEventListener(phase = AFTER_COMMIT) should be used so the alert is only saved if the stock update commits successfully.

Relevant Code

```
// LowStockEventListener.java
@EventListener                      // fires INSIDE the transaction
public void handleLowStockEvent(LowStockEvent event) {
    alertRepository.save(alert);    // alert saved even if stock txn rolls back
}
```

Recommended Fix

Replace @EventListener with @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT) to guarantee the alert is only created after the stock transaction has successfully committed.

Impact

Orphaned LowStockAlerts with no corresponding stock change. Data inconsistency between audit and inventory schemas.

BUG-008 HIGH

H3Helper Uses a Static Initializer — Startup Crash If H3Core Fails, No Null Safety

Component	H3Helper / Warehouse entity
File(s)	*H3Helper.java, Warehouse.java*

Summary

H3Helper initializes H3Core in a static block and wraps the IOException in a RuntimeException. If the H3 native library is missing or fails to load, the entire application context fails to start with a cryptic ExceptionInInitializerError. Additionally, Warehouse.computeH3() calls H3Helper.latLngToH3() in @PrePersist/@PreUpdate, but if H3Helper.h3 is null (initializer failed silently in a test context), a NullPointerException is thrown on every warehouse save.

Steps to Reproduce

1. Run in an environment without the H3 native library
2. Application throws ExceptionInInitializerError wrapping RuntimeException("Failed to init H3Core")
3. In unit tests, H3Helper.h3 may be null causing NullPointerException in Warehouse.computeH3()

Root Cause

Static initialization of a native library is fragile. H3Core.newInstance() is called at class load time with no ability to inject or mock it in tests.

Relevant Code

```
// H3Helper.java
static {
    try {
        h3 = H3Core.newInstance(); // if this fails → ExceptionInInitializerError
    } catch (IOException e) {
        throw new RuntimeException("Failed to init H3Core", e);
    }
}
```

Recommended Fix

Convert H3Helper to a Spring @Component with @PostConstruct initialization. Inject H3Core as a @Bean in AppConfig. This makes it mockable in tests and gives Spring control over the lifecycle.

Impact

Application startup failure in environments without H3 native library. Untestable warehouse persistence logic.

BUG-009 MEDIUM

No Pagination on List Endpoints — Memory and Performance Risk

Component	All Controllers / Services
File(s)	*ProductController.java, StockController.java, WarehouseController.java, AlertController.java*

Summary

All GET list endpoints (getAllProducts, getAll warehouses, getLowStockItems, getAll alerts, getStockByWarehouse, getAll analytics) call repository.findAll() with no pagination, sorting, or filtering. As data grows, these endpoints will load entire tables into memory and serialize them in a single HTTP response.

Steps to Reproduce

1. Insert 100,000 stock records
2. Call GET /api/stock/warehouse/1
3. All 100,000 records returned in one response — high memory usage, slow response, possible OOM

Root Cause

Repositories extend JpaRepository but no Pageable parameters are used. Controllers take no page/size query parameters.

Relevant Code

```
// StockController.java
@GetMapping("/warehouse/{warehouseId}")
public ResponseEntity<List<Stock>> getByWarehouse(@PathVariable Long warehouseId) {
    return ResponseEntity.ok(stockService.getStockByWarehouse(warehouseId)); // no
pagination
}
```

Recommended Fix

Add Pageable parameter to service/repository methods. Accept page and size query parameters in controllers. Return Page<T> or a custom paginated wrapper.

Impact

Memory exhaustion and slow responses at scale. Possible OutOfMemoryError in production.

BUG-010 MEDIUM

Password Returned in Register Response via User Entity

Component	AuthController

File(s)	*AuthController.java, User.java*

Summary

The /api/auth/register endpoint returns Map.of("userId", ..., "username", ..., "role", ...) which is safe. However, if the response is ever changed to return the User entity directly (a common refactoring mistake), the passwordHash field would be serialized. The User entity has no @JsonIgnore on passwordHash, making it one refactor away from leaking BCrypt hashes.

Steps to Reproduce

1. Change register endpoint to: return ResponseEntity.ok(user)
2. Response body now contains: {"userId":1, "username":"alice", "passwordHash":"$2a$10$...", ...}

Root Cause

User entity has no @JsonIgnore or @JsonProperty(access = WRITE_ONLY) on the passwordHash field. No dedicated UserResponse DTO exists.

Relevant Code

```
// User.java — no @JsonIgnore
private String passwordHash; // will serialize if entity is returned directly

// AuthController — currently safe but fragile
return ResponseEntity.ok(Map.of("userId", user.getUserId(), ...));
```

Recommended Fix

Add @JsonIgnore to User.passwordHash. Create a UserResponseDto that explicitly includes only safe fields. Update the controller to use the DTO.

Impact

BCrypt password hashes exposed in API response if entity is returned directly — high risk during future refactoring.

BUG-011 MEDIUM

AnalyticsService.aggregateByH3() Loads Entire Stock Table — O(n) Memory

Component	AnalyticsService
File(s)	*AnalyticsService.java*

Summary

aggregateByH3() calls stockRepository.findAll() to load every stock record into memory, then groups them in Java using streams. For large inventories this is extremely inefficient — the same aggregation (GROUP BY h3Index, COUNT DISTINCT product_id, SUM quantity) can be done entirely in SQL with a single query.

Steps to Reproduce

1. Insert 500,000 stock records across 1,000 warehouses

2. Call POST /api/analytics/aggregate
3. All 500,000 records loaded into heap — response is very slow, high GC pressure

Root Cause

In-memory aggregation using Java streams instead of a SQL GROUP BY query. The InventoryAnalyticsRepository has no aggregation query methods.

Relevant Code

```
// AnalyticsService.java
List<Stock> all = stockRepository.findAll(); // loads entire table!
Map<Long, List<Stock>> grouped = all.stream()
    .filter(s -> s.getH3Index() != null)
    .collect(Collectors.groupingBy(Stock::getH3Index));
```

Recommended Fix

Add a @Query to StockRepository: SELECT s.h3Index, COUNT(DISTINCT s.product.productId), SUM(s.quantity) FROM Stock s WHERE s.h3Index IS NOT NULL GROUP BY s.h3Index. Map results to a projection interface.

Impact

Severe performance degradation and potential OutOfMemoryError as stock data grows.

BUG-012 MEDIUM

spring.jpa.hibernate.ddl-auto=update in Production Configuration

Component	Configuration
File(s)	*application.properties*

Summary

The production application.properties uses spring.jpa.hibernate.ddl-auto=update. This setting auto-modifies the database schema on every application startup. It can add columns and tables but never drops them, can corrupt data during concurrent deployments, and should never be used in production.

Steps to Reproduce

1. Deploy application with ddl-auto=update
2. Rename an entity field — old column remains, data is split
3. Deploy two instances simultaneously — both try to alter the schema concurrently

Root Cause

Development convenience setting left in the default application.properties with no profile separation.

Relevant Code

```
# application.properties
spring.jpa.hibernate.ddl-auto=update  # NEVER use in production
```

Recommended Fix

Use spring.jpa.hibernate.ddl-auto=validate (or none) in production. Manage schema migrations with Flyway or Liquibase. Create separate application-dev.properties and application-prod.properties profiles.

Impact

Risk of schema corruption, data loss, and race conditions during rolling deployments.

BUG-013 MEDIUM

LowStockEvent Fired on Exact Threshold — Duplicate Alerts on Repeated Updates

Component	StockService / LowStockEventListener
File(s)	*StockService.java, LowStockEventListener.java*

Summary

The duplicate-alert guard in LowStockEventListener checks for an existing unresolved alert per product, but ignores the warehouse. If the same product is low-stock in two different warehouses, only the first alert will be created. Subsequent low-stock events for the same product in a different warehouse will be silently dropped.

Steps to Reproduce

1. Product P1 has restockThreshold=5
2. P1 stock in Warehouse A drops to 3 → alert created for (P1, Warehouse A)
3. P1 stock in Warehouse B drops to 2 → no alert created (duplicate check by productId only)
4. Staff is unaware of low stock in Warehouse B

Root Cause

The repository query findByProduct_ProductIdAndIsResolvedFalse() only checks productId, not warehouseId. The intent was per-product deduplication, but the requirement should be per (product, warehouse) pair.

Relevant Code

```
// LowStockEventListener.java
bool alreadyOpen = !alertRepository
    .findByProduct_ProductIdAndIsResolvedFalse(stock.getProduct().getProductId())
    .isEmpty(); // BUG: ignores warehouse — wrong dedup scope
```

Recommended Fix

Add a repository method findByProduct_ProductIdAndWarehouse_WarehouseIdAndIsResolvedFalse() and use it for the duplicate check.

Impact

Missing low-stock alerts for warehouses. Inventory shortfalls go undetected when a product is already alerted in another warehouse.

BUG-014 MEDIUM

AlertController Missing From Security Rules — Unauthenticated Access Possible

Component	SecurityConfig / AlertController
File(s)	*SecurityConfig.java, AlertController.java*

Summary

SecurityConfig defines rules for /api/auth/**, /api/admin/**, /api/warehouses/**, /api/products/**, /api/analytics/**, and /api/stock/**. The /api/alerts/** path is not mentioned. The catch-all .anyRequest().authenticated() does require authentication, but no role restriction is applied. Any authenticated user (including VIEWER) can resolve alerts via PUT /api/alerts/{id}/resolve.

Steps to Reproduce

1. Login as a VIEWER user and obtain JWT
2. Call PUT /api/alerts/1/resolve
3. Alert is resolved — VIEWER should not have this permission

Root Cause

/api/alerts/** was not added to the SecurityConfig authorization rules. Only authentication (not role authorization) is enforced via the anyRequest().authenticated() fallback.

Relevant Code

```
// SecurityConfig.java — /api/alerts is missing
.requestMatchers("/api/auth/**").permitAll()
.requestMatchers("/api/admin/**").hasRole("Admin")
// ... other rules ...
// MISSING: .requestMatchers("/api/alerts/**").hasAnyRole("Admin","Staff")
.anyRequest().authenticated() // too permissive for alert resolution
```

Recommended Fix

Add .requestMatchers("/api/alerts/**").hasAnyRole("ADMIN","STAFF") before the anyRequest() rule.

Impact

Viewer-role users can resolve low-stock alerts, undermining the role-based access control model.

BUG-015 LOW

Warehouse Entity Has Two Duplicate @Table and @Entity Import Blocks

Component	Warehouse entity
File(s)	*Warehouse.java*

Summary

The Warehouse.java file contains duplicate import blocks and duplicate @Entity/@Table annotations. The class declares jakarta.persistence.* twice and the Lombok annotations (@Data, @AllArgsConstructor, etc.) appear twice at the class level. This compiles because Java deduplicates annotations, but it is a defect likely caused by a copy-paste error.

Steps to Reproduce

1. Open Warehouse.java
2. Observe two sets of package import blocks (lines 1-13 and lines 14-16)
3. Observe duplicate Lombok and JPA annotations on the class

Root Cause

Copy-paste error during editing. Duplicate imports and annotations were never cleaned up.

Relevant Code

```
// Warehouse.java — duplicate imports and annotations
import jakarta.persistence.*;
import lombok.*;
// ... then again ...
import jakarta.persistence.*;  // duplicate!
import lombok.*;               // duplicate!

@Entity @Data @AllArgsConstructor // first set
@Entity @Data @AllArgsConstructor // second set (duplicate)
```

Recommended Fix

Remove duplicate import statements and duplicate annotations. Leave one clean set of class-level annotations.

Impact

Code quality and maintainability issue. Potential confusion for static analysis tools and future maintainers.

BUG-016 LOW

JwtUtil Uses Deprecated JJWT API (parserBuilder deprecated in newer versions)

Component	JwtUtil
File(s)	*JwtUtil.java*

Summary

JwtUtil uses Jwts.parser() which is the deprecated pre-0.11.0 API. The inline comment even acknowledges this: '// <- instead of parserBuilder()'. JJWT 0.11+ requires Jwts.parserBuilder().setSigningKey(...).build().parseClaimsJws(token). Using the deprecated API generates compiler warnings and will break on future JJWT upgrades.

Steps to Reproduce

1. Upgrade JJWT to 0.12+
2. Jwts.parser() and .setSigningKey() are removed — compilation fails

Root Cause

Code was written targeting an older JJWT API. The comment in the source acknowledges the issue but it was left unresolved.

Relevant Code

```
// JwtUtil.java
public String extractUsername(String token) {
    return Jwts.parser()              // <- instead of parserBuilder() (deprecated!)
            .setSigningKey(getKey())
            .parseClaimsJws(token)
            .getBody()
            .getSubject();
}
```

Recommended Fix

Migrate to Jwts.parserBuilder().setSigningKey(getKey()).build().parseClaimsJws(token) for JJWT 0.11.x, or to Jwts.parser().verifyWith(secretKey).build().parseSignedClaims(token) for JJWT 0.12+.

Impact

Compiler deprecation warnings. Breaking change on JJWT version upgrade.